

Geometry that turns a target into joint angles

M8 • 02 May 2027 - 03 Jun 2027 • 75 hours

Outcome of the month

Create a verified kinematics library for a planar two-link arm and a leg with three degrees of freedom. It will answer four questions: where the mechanism's endpoint lies for given angles, which angles reach a target point, how joint velocities relate to endpoint velocity, and how an external force relates to joint torques. This directly prepares for moving a walking robot's foot and a fingertip. Deliver code, a geometry specification, visualisations, automated checks and clear explanations of inverse-kinematics failure.

What you should already understand

Prerequisites from earlier months are vectors, matrices, trigonometry, derivatives, Python, NumPy and tests. Bring the CAD leg's dimensions and axes from M5 and the available joint's sign, zero offsets and angle ranges from M7. Before starting, manually evaluate $\sin$ and $\cos$ at 0 , $\pi/2$ and π ; multiply a rotation matrix by a basis vector; convert millimetres to metres. Even if a numerical library performs everything, explain the result's direction on a drawing. The main errors this month usually arise from inconsistent conventions rather than difficult equations.

The first reference is a planar 2R mechanism: two revolute joints, lengths l_1 and l_2 , and motion in the x-y plane. The second is an exercise yaw-pitch-pitch leg: base rotation about z and two bends in a radial vertical plane. If the M5 CAD model uses a different first joint axis, such as abduction, write a separate transform chain from CAD; do not insert its dimensions into another mechanism's formula. Reference models verify the mathematics, while adaptation to the actual leg must retain its real axes and offsets.

What M8 does not require

Kinematics describes geometry, but does not establish sufficient torque, body stability or collision freedom for every part. First calculate and visualise commands; permit transmission to a real actuator only through M6-M7 limits and after a separate rig check. A complete gait planner, force control and dynamic balance belong to later modules. For a hand, 2R can model two phalanges, but tendon transmission and object contact require additional models.

At the end, check forward/inverse kinematics (FK/IK) consistency, verify the Jacobian numerically and ensure predictable failure for unreachable targets. Declare numerical-test tolerances separately from actual mechanical accuracy: a small computational residual does not remove backlash, encoder-zero error or incorrect dimensions.

Project structure and 75 hours

Allocation across four blocks

Block A, 1-20: coordinate frames, rotations and transforms, 10 hours theory and 10 practice. Block B, 21-40: link models and forward kinematics for 2R and 3-DOF, 10+10. Block C, 41-60: analytical and numerical inverse kinematics, 10+10. Block D, 61-75: Jacobians, force, singularities and acceptance checks, 5+10. Total: 35 hours of study and analysis, 40 hours of implementation and experiments. Theory includes your own derivations on paper; practice includes code, plots, tests and investigation of specific errors.

Divide ten-hour theory blocks into 2-hour sessions: one mechanism, one hand-worked example and one software check. Schedule ten-hour practical blocks on Sundays and divide them into small stages; breaks add to calendar time. Split the last 5 theory hours 2+2+1. Calendar boundaries from the overall plan may overlap adjacent modules; retain rest days, holidays and integration reviews. Allocate the hours below to available sessions without counting them again in the neighbouring month.

Library organisation

Create geometry/ for transforms, models/ for 2R and the leg, solvers/ for IK, validation/ for reference poses, visualizations/ for drawings and reports/ for results. Begin with a clear Python/NumPy implementation. Add a C++ core only for a small, already verified interface if time allows; a Rust port is optional. For every array, state shape, units and coordinate frame. Do not give two different types of point the same name p without specifying their origin and the basis in which they are expressed.

Conventions document

In conventions.md, record right-handed frames, body and joint axis directions, positive angle, length and angle units, quaternion ordering, the meaning of T_{AB} , multiplication order and chosen IK branch. Recommended convention: T_{AB} transforms coordinates from B to A, so $p_A = T_{AB} \cdot p_B$ for a homogeneous point. Check every CAD export and library adapter against this document. Do not fix an inverted leg by inserting an arbitrary minus sign before finding the cause.

For every experiment, save mechanism parameters, inputs, expected result and random-configuration generator seed. Use error statuses UNREACHABLE, LIMIT_VIOLATION, SINGULAR, NO_CONVERGENCE and INVALID_INPUT. Never return the final numerical approximation as a successful command without verification: residual and status are part of the API result.

Coordinate frames and composition of motion

What to study about transforms

Rotation R specifies basis orientation, and translation p locates its origin. A homogeneous matrix T combines them: the upper block contains R and p , and the bottom row is $[0,0,0,1]$. For a pure rotation, $R^T R = I$ and $\det(R) = +1$. A matrix with $\det = -1$ is a reflection, not an admissible rigid-body rotation. Under the chosen convention, $T_{AC} = T_{AB} \cdot T_{BC}$. Transform order matters: rotating a local vector and then translating the result is generally different from reversing these operations. Modern Robotics §3.3.1 establishes notation; follow frame indices rather than the words “left” and “right”.

Task 1. Foot coordinates in different bases

Inputs: a drawing of body, hip and foot with simple known offsets. 1. Define $T_{\text{body_hip}}$ and foot coordinates in hip. 2. Calculate body-frame coordinates by hand. 3. Verify with the matrix in code. 4. Apply the inverse transform and recover the original point. Deliver an axis drawing and tests for several poses, including a $\pi/2$ rotation. Check that a rigid transform preserves distance between two points. Edge case: incorrect multiplication order must fail a test with nonzero translation and rotation, even if the case without rotation appears correct.

Task 2. Translation of a point and a direction

The aim is to avoid adding hip position to a force or velocity vector. Take a contact point and force direction in one frame. 1. Transform the point including translation. 2. Transform the free vector using rotation only. 3. Compare results when base position changes without a change in orientation. 4. Add separate `transform_point` and `rotate_vector` functions, or types that prevent confusion. Save the test and illustration. Check that shifting the coordinate origin changes point coordinates but not a free vector's components when the basis is unchanged.

Active and passive interpretations

A matrix may rotate a physical object or describe that same object in another coordinate frame; either use needs consistent frame indices and inverse-matrix definitions. Draw two diagrams: body rotation relative to the world, and a stationary world point expressed in the rotated body's frame. Label what physically moves. For a leg, this explains why holding a foot fixed in the world while tilting the body changes the local target point. Whole-robot IK is not yet needed; demonstrating the correct transformation of one point is sufficient.

Rotations, quaternions and the mechanism tree

Essential SO(2) and SO(3)

SO(2) describes planar rotations and SO(3) spatial rotations. Learn basic rotation matrices and composition, then axis-angle orientation and a normalised quaternion. Euler angles are readable but depend on axis order and have degenerate configurations. Quaternions q and $-q$ describe the same orientation; comparing components as ordinary points is insufficient. Record the chosen order, for example $[x,y,z,w]$ in your adapter, and check every external library's requirements, including SciPy Rotation.

Task 3. Verify orientation representations

Use identity orientation, a $\pi/2$ rotation about each axis and a compound rotation. 1. Write expected axis directions by hand. 2. Convert matrix to quaternion and back. 3. Check orthogonality, determinant and angular error. 4. Supply q and $-q$ and confirm the physical result is identical. Deliver reference orientations and an adapter test. Edge cases: a zero quaternion, NaN and a strongly non-unit input. The API must reject or normalise it according to an explicit policy, without concealing invalidity.

Task 4. Coordinate-frame tree of the CAD leg

For each M5 part, specify parent, joint origin, axis, joint type, zero position and fixed offset. 1. Draw the tree from body to foot. 2. Separate fixed mounting transforms from rotations q . 3. Assemble the chain and visualise every local axis. 4. Check the zero pose and move each joint individually. Save geometry in machine-readable form and images of at least four poses. Check that one joint moves only its descendants; the body and parent links stay fixed. Error: an axis exported in world coordinates must not be treated as local without transformation.

Two ways to describe one chain

Understand what a Denavit-Hartenberg table specifies and why standard and modified versions use different conventions. An explicit transform chain matching CAD is convenient for the first implementation. Then study the product of exponentials: fixed screw axes and the home configuration define FK through a product of exponentials. Derive one planar revolute example and compare with explicit matrices. Do not rewrite the entire project in three notations; the goal is to read other models and recognise differences in conventions.

Forward kinematics of the arm and leg

Task 5. Planar 2R arm

For links l_1, l_2 and relative angles q_1, q_2 in the x-y plane, derive $x = l_1 \cdot \cos(q_1) + l_2 \cdot \cos(q_1 + q_2)$, $y = l_1 \cdot \sin(q_1) + l_2 \cdot \sin(q_1 + q_2)$. The second link's world angle is $q_1 + q_2$, not q_2 . 1. Derive the formula from a drawing. 2. Implement analytical FK and an independent matrix chain. 3. Compare straight, bent and folded poses. 4. Build a grid of permissible angles. Deliver two implementations and a reachable-point map. Check that at $q_1 = q_2 = 0$ the endpoint is $(l_1 + l_2, 0)$; with equal lengths and $q_2 = \pi$, it reaches the base. Handle zero and negative lengths according to geometry-validation rules.

Task 6. Exercise leg with three axes

For yaw-pitch-pitch, define q_1 about z and q_2, q_3 as angles in the vertical plane, with positive pitch raising the link. Let l_0 be a fixed radial offset and $r = l_0 + l_1 \cdot \cos(q_2) + l_2 \cdot \cos(q_2 + q_3)$. Then $x = r \cdot \cos(q_1)$, $y = r \cdot \sin(q_1)$, $z = l_1 \cdot \sin(q_2) + l_2 \cdot \sin(q_2 + q_3)$. 1. Draw this exact mechanism. 2. Build an independent transform chain. 3. Compare both forms at random permissible angles. 4. Add $T_{\text{body_hip}}$ and find foot coordinates in the body frame. Save a 3D view of the leg with axes and tests. If CAD uses another mechanism, use its own chain and a separate reference model.

Comparison with CAD and a library

Choose at least five numerically specified poses and measure endpoint position in CAD or the saved geometric representation. Compare values with your implementation and Robotics Toolbox or an independent matrix implementation. State what is compared in the table: coordinates, units and basis. A large error may arise from millimetres instead of metres, another joint zero, axis ordering or a different point on the part. Find the cause one transform at a time from root to endpoint, rather than compensating for the final error with an arbitrary offset.

Accuracy checks

For a purely numerical reference, choose tolerance in advance relative to mechanism length and machine precision; for example, around 10^{-6} m may be a reasonable starting target for a metre-scale model in double precision, but it is not the robot's mechanical accuracy. Check geometry scaling and errors at boundary angles. Save the maximum norm of the difference between implementations and the poses where it occurs. Random checks supplement, rather than replace, hand-worked reference poses.

Inverse kinematics and solution branches

Analytical solution for the planar arm

For target (x,y) , define $c2=(x^2+y^2-l_1^2-l_2^2)/(2l_1l_2)$. If $|c2|$ is substantially greater than 1, the target is unreachable; a small round-off excess may be clipped only within the declared numerical tolerance. For both branches, obtain $q2=\text{atan2}(\pm\sqrt{1-c2^2},c2)$, then $q1=\text{atan2}(y,x)-\text{atan2}(l_2\sin(q2),l_1+l_2\cos(q2))$. Geometrically reachable points lie between circles of radii $|l_1-l_2|$ and l_1+l_2 , but joint limits further reduce the workspace. Degenerate lengths and folded configurations need explicit handling rules, rather than division by zero.

Task 7. Both branches and joint limits

1. Select a target with two permissible solutions. 2. Calculate both branches and check FK for each. 3. Add angle ranges from geometry and M7; remove solutions that violate them. 4. Choose a solution close to the previous q while respecting the permitted range, rather than merely the shortest angular difference around a circle. Deliver an API returning permissible solutions and a failure reason. Check that $\text{FK}(\text{IK}(p))$ is close to p for all accepted points; $\text{IK}(\text{FK}(q))$ need not return the original branch. Edge case: a target on the outer boundary where the two branches coincide.

Task 8. Foot targets for the 3-DOF leg

For the exercise yaw-pitch-pitch leg, first calculate $q1=\text{atan2}(y,x)$, then solve the planar problem at $(\sqrt{x^2+y^2}-l_0,z)$. This branch assumes a selected radial-projection direction; the model must explicitly bound additional configurations and ranges. 1. Transform the target from body to hip coordinates. 2. Calculate solutions and check limits. 3. Verify them with full FK. 4. Follow a short foot-lift-and-lower trajectory while retaining the selected branch. Save animation, angles and maximum geometric residual. At $x=y=0$, yaw is undefined: return a special status or retain a permissible previous yaw according to the interface rules.

A failure must not become a command

Distinguish no geometric solution, range violation and numerical failure. Return status, $q_candidates$, residual, chosen branch and a diagnostic message. Do not silently replace an unreachable point by the nearest one: projection must be a separate explicitly requested mode returning the target actually reached. For a robot, the difference matters: an unnoticed change in support point changes body position, and a fingertip may fail to reach the object.

Numerical IK and library design

Task 9. Iterative approximation

The aim is to solve a kinematic problem whose analytical formula is inconvenient and learn to diagnose convergence. First use the familiar 2R model as a reference. Error $e = p_{\text{target}} - FK(q)$. Linearisation gives $e \approx J \cdot \Delta q$; solve the system or use a pseudoinverse instead of explicitly inverting a matrix. For damped least squares, $\Delta q = J^T \cdot (J \cdot J^T + \lambda^2 I)^{-1} e$, where λ controls attenuation of large steps near degeneracy. Implement with solve, a step limit and a finite iteration count. Do not present this method as a guarantee of a global solution.

1. Begin with a central reachable target and several initial q values. 2. Record error norm and step size at each iteration. 3. Add joint limits and a termination condition for lack of improvement. 4. Compare with analytical IK on the same targets. Save convergence plots and a table of successful and failed runs. Return SUCCESS only after checking final FK and limits. Edge cases: unreachable target, poor initial seed, nearly straight arm and maximum iteration count. Do not return the last approximation with a large residual as an executable command.

Unified robot-kinematics v1 interface

The model needs `forward(q)`, `inverse(target,seed,limits)`, `jacobian(q)`, `validate(q)` and a geometry description. Inputs must not be changed silently. `forward` returns position and, when required, orientation with its basis specified. `inverse` returns a structured result rather than an unexplained NaN array. Specify the input coordinate frame explicitly. Use one reference dataset for the analytical implementation, numerical method and library comparison. A C++ wrapper may follow these checks; porting to three languages is not the month's goal.

A small integration scenario

Place the body in several known poses and define one stationary world support point. Transform that point to the hip frame, solve IK and recover world foot coordinates. Then repeat for the 2R finger: a small target-point contour in its working plane. Plot target, reached point, angles, status and distance to limits together. This checks geometric consistency across components. Velocities and torques require separate calculations; a smooth-looking animation does not establish physical feasibility of the profile.

Jacobians, forces and singular configurations

Task 10. Calculate foot velocity from joint velocities

For position FK, matrix J contains partial derivatives of coordinates with respect to q ; each column describes endpoint motion from a small motion of one joint. Linear velocity $v=J(q)\cdot\dot{q}$. Derive J for 2R by hand, then obtain it for the leg analytically or using rotation axes and the vector to the foot expressed in one coordinate frame. Begin only with the 2×2 or 3×3 position Jacobian; a spatial velocity (twist) has 6 components and its Jacobian is $6\times n$. Specify linear/angular component ordering separately.

Central-difference check: column i is approximately $[FK(q+h\cdot e_i)-FK(q-h\cdot e_i)]/(2h)$. Compare several h values, for example 10^{-4} , 10^{-5} and 10^{-6} rad, at random and boundary poses. Save maximum error norm and a plot against step size. An excessively small h amplifies round-off, while an excessively large h increases linear-approximation error. Then specify $\dot{q}$ and compare predicted endpoint velocity with a small actual FK step. Edge case: derivative expressed in the wrong basis with a nonzero base rotation.

Task 11. Foot force and joint torque

Fix the meaning of F : external force applied at the mechanism's endpoint, expressed in the same basis as J . The corresponding generalised torque is $\tau=J^TF$; to balance this load, the actuator generally provides the opposite contribution while accounting for other forces. 1. Check one joint using $r\times F$. 2. Compare with J^TF for the arm. 3. Repeat for the leg and several force directions. 4. Check virtual work, $\tau^T\delta q\approx F^T\delta p$. Deliver a static-load calculation and sign test. The formula itself excludes link mass, acceleration and friction; M9 adds these.

Task 12. Map proximity to singularity

A singularity is a loss of Jacobian rank relative to its normal maximum. For 2R, compare bent and fully straight poses. Over a grid, calculate the smallest singular value and available velocity directions; state normalisation and units explicitly. Request motion along the direction lost in the straight pose. Save the map and explain why a small target displacement may require a large Δq . Add status checks and step limits to IK. Proximity to singularity does not itself mean collision, but requires special command-handling rules.

Work sequence: hours 1-40

Hours 1-10: conventions and manual checks

1-2: review vectors, radians and matrix rotations; create conventions.md. 3-4: study homogeneous transforms and work a body/hip example by hand. 5-6: understand active and passive transform meanings and the point/vector distinction. 7-8: study $SO(2)$, $SO(3)$, axis-angle and quaternion representations; fix component ordering. 9-10: draw the CAD leg tree and initial axis positions. At every stage, calculate at least one reference without a ready-made robotics library; adding one later provides an independent comparison.

Hours 11-20: geometry practical

11-12: implement transforms and task 1's coordinate round trip. 13-14: perform task 2, separate points and directions and add a negative test. 15-16: verify task 3 orientation representations and invalid-input handling. 17-18: build the task 4 tree and visualise local axes. 19-20: compare four known poses with a drawing or CAD, save references and correct conventions before continuing. Do not proceed to IK with even one unknown axis direction; its error becomes harder to diagnose there.

Hours 21-30: derive forward kinematics

21-22: derive 2R equations geometrically and check zero angles. 23-24: derive the 2R matrix chain and compare with the formula. 25-26: study yaw-pitch-roll and its difference from the actual CAD leg. 27-28: explore DH and the product of exponentials; reproduce one revolute example. 29-30: define reference poses, tolerance scales and comparison rules for an external library. Separately list fixed offsets that must not be confused with link lengths.

Hours 31-40: two verified models

31-32: implement task 5 and the planar arm workspace grid. 33-34: implement the task 6 exercise-leg FK and body transform. 35-36: adapt the chain to actual M5 geometry if axes differ, retaining separate reference-model tests. 37-38: compare five poses with CAD and one independent implementation. 39-40: run random tests, check units and export, and freeze baseline-fk. Finish every step by saving verification results; correcting a discovered geometric error belongs within this block and takes priority over animation styling.

Work sequence: hours 41-75

Hours 41-50: the inverse problem

41-42: derive analytical 2R IK from the triangle, including both branches. 43-44: study reachability, joint limits and the convention for selecting a nearby solution. 45-46: reduce the exercise leg to a planar problem; handle yaw and offset IO separately. 47-48: study linearisation, numerical IK and the initial guess using Modern Robotics §6.2. 49-50: prepare the result format, validation rules, stopping criteria and unreachable/boundary targets. By now, define how the program will fail before writing its main iteration loop.

Hours 51-60: targets become angles

51-52: perform task 7 for both branches; check FK/IK round trips back to the target. 53-54: perform task 8 and save the foot trajectory and body-frame position. 55-56: implement task 9 numerical IK for 2R and compare different seeds. 57-58: add limits, step bounds and failure diagnostics; test the leg's actual chain. 59-60: assemble the unified API and world-point holding scenario while the body moves. Commands remain geometric data at this stage; SUCCESS does not automatically authorise physical motion.

Hours 61-65: velocity, force and singularity

61-62: derive the arm Jacobian and geometric column for a revolute joint; study numerical derivatives. 63-64: study $\tau = J^T F$, virtual work and rank loss; select verification metrics. 65: prepare the test-pose map and final scenarios, including nonnumeric inputs and inconsistent coordinate frames. Split these five hours 2+2+1. The month's aim is understandable calculations for small mechanisms, rather than a deep proof of every Lie-group property.

Hours 66-75: verify the library

66-67: perform task 10 and compare Jacobians with finite differences for several h values. 68-69: check velocities and task 11 virtual work and external-force signs. 70-71: build the task 12 map and investigate IK near singularity. 72-73: perform every fault-checking scenario and compare an independent library with your code. 74-75: assemble README, plots, reference data and a short demonstration of both mechanisms. If time is short, omit the second language and decorative visualisation; both reference models, FK/IK consistency, Jacobian verification and correct failure behaviour are compulsory.

Acceptance, negative tests and self-check

Required scenarios

F1: rotation and translation applied in reverse order. The task 1 reference pose detects it. F2: millimetres treated as metres. Scale checking and a known length reveal the discrepancy; relative error between functions sharing the same mistake must not produce a false PASS. F3: target beyond the outer radius or inside the unreachable inner circle of 2R. Return UNREACHABLE. F4: geometrically reachable point requires a forbidden angle. Return LIMIT_VIOLATION. F5: fully straight arm. The Jacobian loses rank and IK must not return an unbounded jump as a permissible command.

F6: two neighbouring requests each have two solution branches. The continuity policy retains a permissible branch or reports the need to switch. F7: poor seed for numerical IK. At the iteration limit, return NO_CONVERGENCE with residual. F8: NaN, zero length or zero quaternion. Reject input according to interface rules. F9: q and $-q$. The orientation check treats them as equivalent. F10: force expressed in a different basis. The virtual-work test detects inconsistency. F11: incorrect encoder-zero sign. Testing the joint adapter in a known calibration configuration reveals the error.

Completion conditions

Both models have documented geometry and automated tests. For accepted IK solutions, the norm of $FK(q)$ -target lies within the declared numerical tolerance and joint angles remain in range. State maximum Jacobian error relative to numerical differentiation with the method, steps and pose set. Include hand-worked references, random checks and independent comparison; two functions with a shared error do not sufficiently validate each other. Unreachable and singular requests return clear status, and reproducing results does not require manual file corrections.

Review questions and handover to M9

Explain what T_{AB} means; why a point and force transform differently; why 2R q_2 is relative; why IK need not recover the same branch; how numerical convergence is defined; why rank matters more than one large J component; what sign the torque balancing an external force must have; and how geometry error differs from backlash. Show answers in your drawings and plots. Pass lengths, local axes, zero offsets, FK, Jacobians and permissible configurations to M9. Masses, centres of mass, inertias and contacts will be added separately; do not conceal their absence behind a successful kinematic animation.

Assigned reading: a focused reading route

Books and notes may be in English; keep the plan, definitions and your explanations in English for this edition. Read only the specified sections within the existing 75 hours.

Core reading and one reference route

1. Core: K. Lynch, F. Park, Modern Robotics. §3.3.1 homogeneous transforms; §4.1.1 product of exponentials; §5.1-5.3 Jacobian, statics and singularities; §6.2 numerical IK. This is the route for tasks 1-12: read the relevant section before deriving equations yourself.

2. Reference: S. LaValle, Planning Algorithms. §3.2 Rigid-Body Transformations and §3.3 Transforming Kinematic Chains. For tasks 1-4, compare the geometric explanation of transforms and chains; one 2D chain and the transition to 3D are sufficient.

3. Core, selectively: R. Tedrake, Robotic Manipulation: Basic Pick and Place. Named sections Forward kinematics and Differential inverse kinematics. For tasks 9-10, connect endpoint coordinates with joint velocities; subsequent motion planning is outside this module.

Documentation for the practical work

SciPy Rotation - apply, inv, from_quat and as_quat; check component ordering and q/-q equivalence in task 3.

SciPy least_squares - bounds, jac, tolerances and status; an independent numerical-ik check.

Robotics Toolbox: DH models - optional reference for fkine, jacob0 and DH conventions.

Complete the reading with an action

Finish reading by drawing axes, deriving a formula and checking one known pose. Do not transfer CAD leg dimensions to yaw-pitch-roll without checking axes and zeros. Separately explain why successful IK, reachability and permissible real motion are different checks. You do not need to read all books in full this month.

Glossary: terms and definitions

Coordinate frame. An origin and ordered axes relative to which point coordinates or physical-vector components are expressed.

Pose. A combined description of body position and orientation relative to an explicitly named coordinate frame.

Rotation matrix. A matrix R preserving lengths and basis handedness; for spatial rotation, $R^T R = I$ and $\det(R) = 1$.

Homogeneous transform. A 4×4 matrix combining rotation and translation of a point between two specified coordinate frames.

Composition. Sequential application of transforms; frame indices determine matrix multiplication order, which generally affects the result.

Degree of freedom. An independent parameter needed to specify a mechanism's configuration under its geometric constraints.

Configuration. A set of independent mechanism coordinates; for the open chain considered here, the joint-angle vector q .

Kinematic chain. Rigid links connected by joints whose relative transforms determine the position of each successive link.

Home configuration. A predefined configuration at zero joint coordinates, serving as the geometric reference.

Joint axis. An oriented geometric line of rotation or direction of translation, expressed in a specified coordinate frame.

Forward kinematics. Calculating a selected link's pose from joint configuration and fixed mechanism geometry.

Inverse kinematics. Finding configurations that achieve a specified pose or point, accounting for joint limits and solution accuracy.

IK branch. One of several distinct configurations or continuous configuration families that achieve the same target pose.

Workspace. The set of reachable points or poses of a selected mechanism endpoint under specified geometry and joint limits.

Quaternion. A four-component, unit-norm orientation representation; quaternions of opposite sign describe the same rotation.

Jacobian. A matrix relating joint velocities locally to selected endpoint coordinates; its dimensions and units depend on the task.

Singularity. A configuration with loss of Jacobian rank in which some endpoint-motion directions become locally unavailable.

Pseudoinverse. A generalisation of matrix inversion for linear least-squares problems that selects a minimum-norm solution.

Damped least squares IK. A regularised local solution that suppresses excessive joint-coordinate changes near degeneracy at the cost of residual error.

Check the relationships: $\dot{p} = J(q)\dot{q}$; with consistent signs, the generalised force from an external action is $J^T F$. Do not confuse it with a holding command.

Equipment and environment for kinematics

NO NEW PURCHASES REQUIRED

Use the existing MacBook Pro M4, CAD and measured M5 dimensions, and M7 calibrations and one rig joint. The existing 3D printer is needed only for fixture checks if necessary; select a profile for its exact model. A complete leg and additional actuators are unnecessary for this mathematics.

PREPARE FOR HOURS 1-20

Save a geometry description: parent/child, joint origin, unit axis, joint type, angle sign, zero, limits, foot point and units. Specify that axis is expressed in the joint frame. Explicitly convert CAD exports from millimetres to metres; use the same conventions when creating URDF later.

Keep the reference 2R, reference yaw-pitch-roll and actual CAD chain separate. Changing lengths does not turn one kinematic architecture into another. For physical illustration, reuse the M7 rig only after checking commands in the model.

INSTALL NOW

No new mandatory packages. On macOS ARM, use pinned Python, NumPy, SciPy, Matplotlib and pytest from M2-M7; SciPy already includes Rotation and least_squares. The existing editor and project JSON/YAML suffice for geometry descriptions. ROS, RViz and Gazebo enter later in the programme.

OPTIONAL, IF NEEDED

If an additional library FK reference is useful, add robotics-toolbox-python in a separate environment using the [official Introduction instructions](#). First check dependencies on macOS ARM or the chosen Ubuntu 24.04 ARM64. The main route uses your own matrix FK and SciPy; a MATLAB licence is unnecessary.

INSTALLATION AND INPUT CHECKS

From a clean directory, import the libraries, print versions, perform a $\pi/2$ rotation and its inverse, and compare q with $-q$. Then check FK at the 2R zero pose, FK/IK round trips back to the target, joint limits and the Jacobian using central differences. Save a three-pose plot and a machine-readable pytest report.

For independent comparison in task 5, use Robotics Toolbox or a separate implementation based on saved transforms; for task 9, use SciPy least_squares. Preparation and checks replace duplicated setup within the 75 hours.

Motion fundamentals: from a point to joints

Required worked example: FK → IK → velocity → force

Use two rigid links in the planar part of a leg with a fixed base. x points right, y up and $+z$ towards the viewer; angles are anticlockwise. Measure q_1 from $+x$ and q_2 relative to the first link. Check $200\text{ mm}=0.2\text{ m}$. Here $v=\dot{q}$, in rad/s; endpoint velocity $\dot{p}$, in m/s, is a different quantity.

For $l_1=l_2=0.2\text{ m}$, add link vectors: $x=0.2\cos q_1+0.2\cos(q_1+q_2)$, $y=0.2\sin q_1+0.2\sin(q_1+q_2)$. At $q=(0;\pi/2)$, endpoint $p=(0.2;0.2)\text{ m}$. FK applies known rotations in sequence; IK solves the inverse geometric problem. The cosine rule gives $\cos q_2=(x^2+y^2-l_1^2-l_2^2)/(2l_1l_2)=0$. The branches are $(0;\pi/2)$ and $(\pi/2;-\pi/2)$; both reach the point, but joint limits may exclude one.

The Jacobian: local motion and its limits

Rotation of joint i moves the endpoint tangentially. Thus a position-Jacobian column is $z_i \times (p-o_i)$: axis z_i and the vector from joint o_i to the endpoint are expressed in one basis. At our pose, $J=[[-0.2; -0.2]; [0.2; 0]]\text{ m/rad}$. With $v=(0.5;-0.25)\text{ rad/s}$, $\dot{p}=Jv=(-0.05;0.10)\text{ m/s}$. This is an instantaneous relation; a finite path requires recalculating J as q changes.

At the straight pose $q=(0;0)$, both tangents are vertical, the x row of J is zero and rank is 1. Instantaneous velocity along x is unavailable, although a bent chain can change x . This is a singularity: a small requested point displacement does not guarantee small joint velocities. IK step limiting and damping do not create the missing direction.

Force uses the same geometry; dynamics comes later

In a separate static experiment, apply $f_{\text{ext}}=(0;-5)\text{ N}$ at the endpoint: this is the environment's force on the robot. With link masses and friction excluded, $\tau_{\text{ext}}=J^T f_{\text{ext}}=(-1;0)\text{ N}\cdot\text{m}$, and holding requires $\tau_{\text{act}}=-\tau_{\text{ext}}=(1;0)\text{ N}\cdot\text{m}$. Torque at the second joint is zero because the force's line of action passes through its axis. Link weight and acceleration change the requirement according to the M9 model; successful IK does not establish motion feasibility.

One check connects tasks 5, 7 and 10-12

Save a drawing of both branches, a p/J value table and tests. Compare J columns with central differences of FK at $h=10^{-4}$ and 10^{-6} rad , then repeat at the straight pose. Independently calculate each torque using $r \times f_{\text{ext}}$. Check power at the stated v : $f_{\text{ext}}^T \dot{p} = \tau_{\text{ext}}^T v = -0.5\text{ W}$. This explains the transpose of J and the external-action sign; it is a calculation, not a torque command to a position-controlled servo.

Included in the 75 hours: 1 hour from 21-22 for this reference FK/IK example; 1 hour from 63-64 for force and singularity; 2 hours from 66-69 for existing Jacobian and virtual-work tests. Retain all other tasks; no new laboratory sessions are added.

Read selectively: Lynch/Park, [Modern Robotics, chapters 4 and 6: introductory FK and Analytic Inverse Kinematics](#); [§5.2 Statics of Open Chains](#). Study the $6 \times n$ spatial Jacobian and 6-component twist after the position Jacobian J .